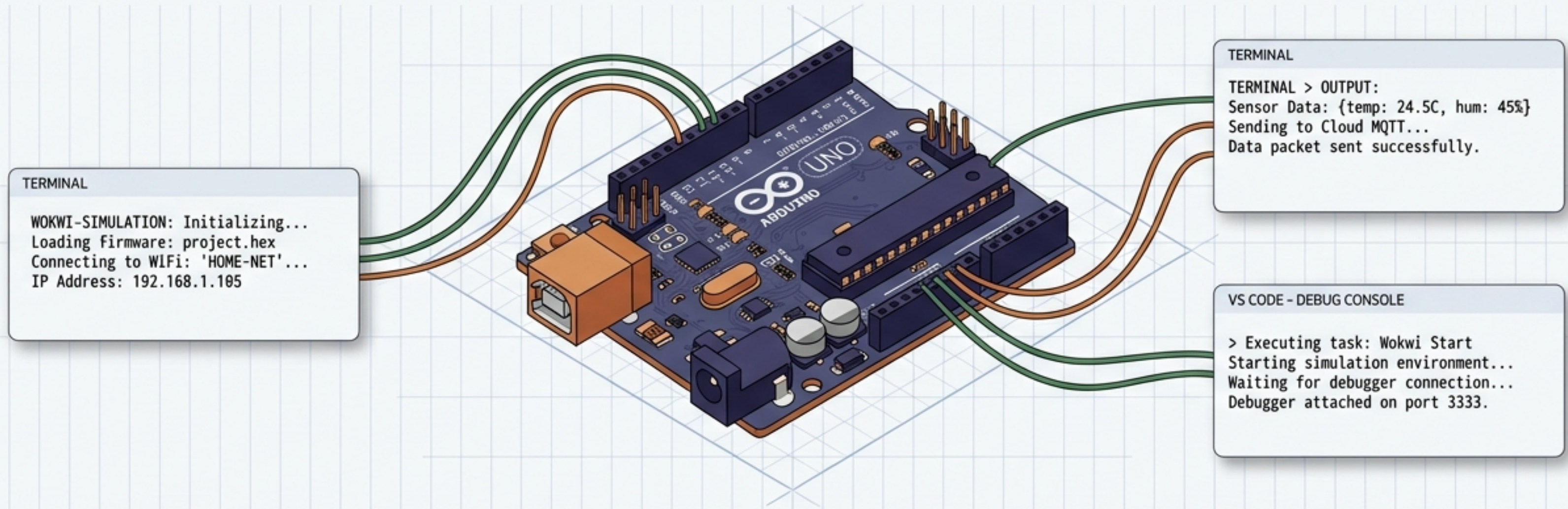


แนะนำการใช้งาน VS Code IDE + Wokwi

แผนสำหรับการจำลองฮาร์ดแวร์ IoT บนเครื่องคอมพิวเตอร์



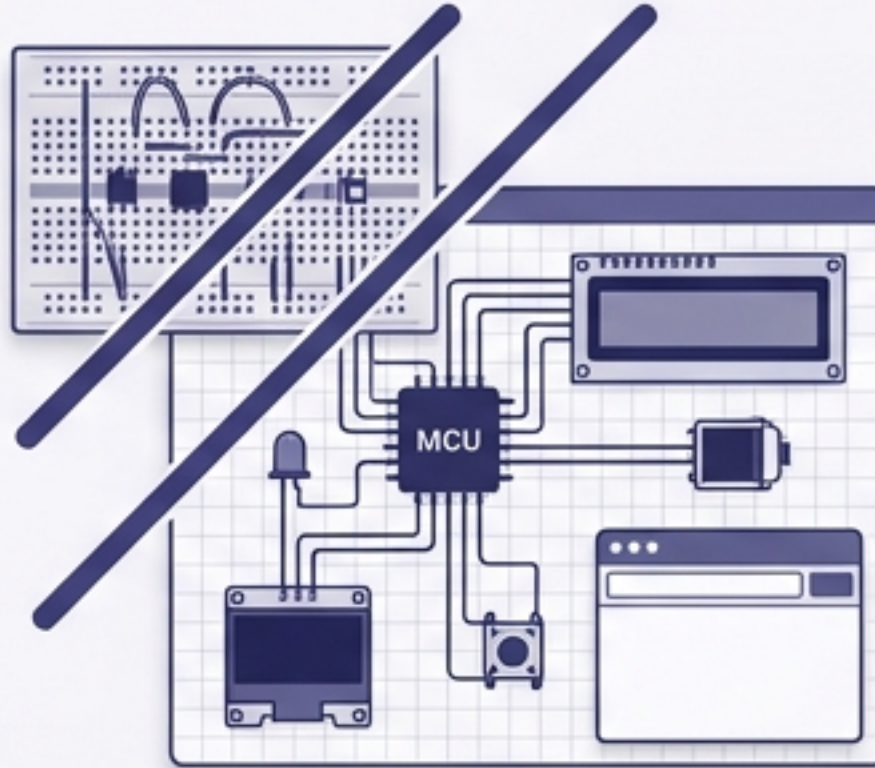
สร้าง คอมไฟล์ และจำลองการทำงานของไมโครคอนโทรลเลอร์
ทั้งแบบ Bare-metal และ Framework โดยไม่ต้องเผาอุปกรณ์จริงแม้แต่ชิ้นเดียว

Wokwi คืออะไร?



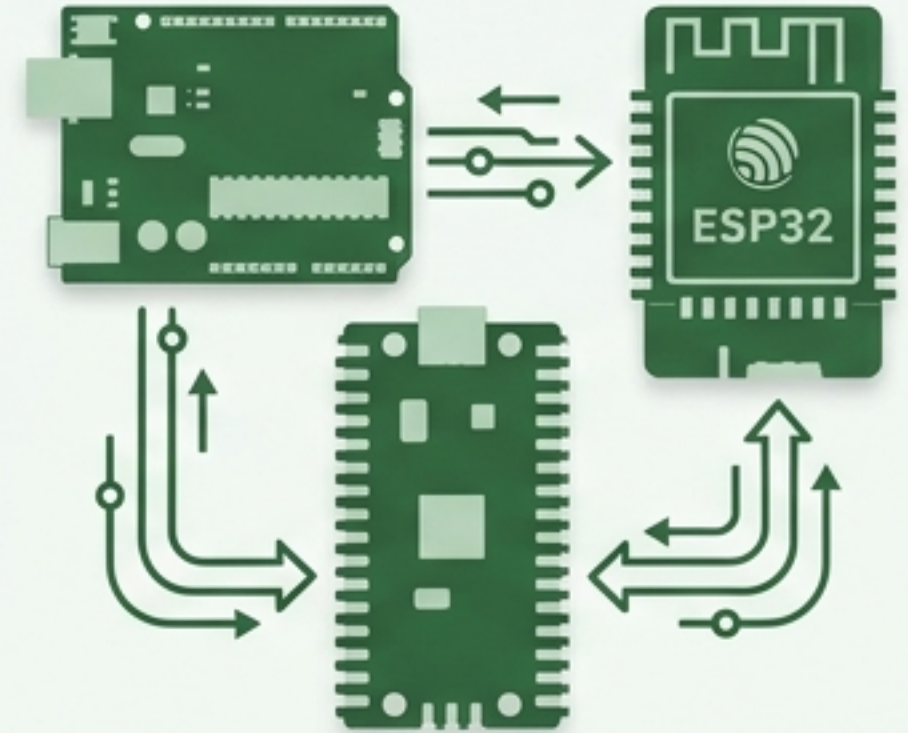
เครื่องมือจำลองออนไลน์และออฟไลน์

เริ่มต้นจากเว็บแอปพลิเคชัน ปัจจุบันสามารถทำงานแบบออฟไลน์ (Offline) ภายใน VS Code เพื่อคอมไพล์โค้ดในเครื่อง



ไม่ต้องใช้ฮาร์ดแวร์จริง

จำลองการทำงานของ MCU และอุปกรณ์ต่อพ่วง (LED, LCD 16x2 I2C, OLED, ปุ่มกด) หมดปัญหาอุปกรณ์พังหรือสายไฟยุ่งเหยิง พร้อม Serial Monitor ในตัว



รองรับหลายภาษาและหลายบอร์ด

รองรับภาษา Arduino (C/C++), MicroPython, Rust และ Verilog จำลองบอร์ด Arduino (Uno, Nano, Mega), ESP32, Pi Pico และ STM32 Nucleo

โครงสร้างราคาและการเข้าถึงบริการ

แผนชุมชน
(Community Plan)

ฟรี!

สำหรับงานส่วนตัวและ
โปรเจกต์โอเพนซอร์ส
สร้างโปรเจกต์สาธารณะ
ได้ไม่จำกัด

งานอดิเรก
(Hobby & Hobby+)

สำหรับผู้ใช้งานส่วนตัว
ขั้นสูง มีระบบสมัครสมาชิก
รายเดือน

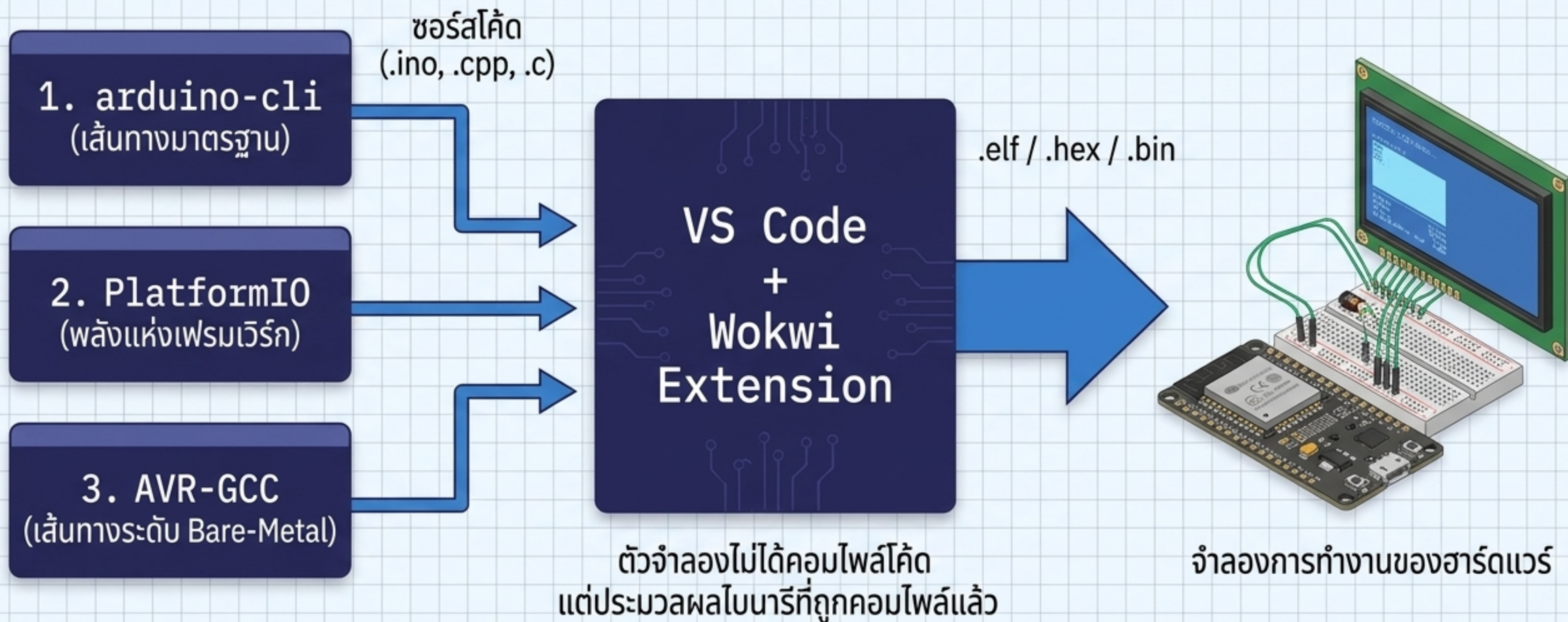
ระดับโปร
(Pro)

สำหรับโปรเจกต์เชิง
พาณิชย์ หรือผู้ใช้ที่
ต้องการสมรรถนะการ
จำลองขั้นสูง

ห้องเรียนมหาวิทยาลัย
(University
Classroom)

สำหรับการศึกษาและ
สถาบันต่างๆ

สถาปัตยกรรมห้องปฏิบัติการเสมือน



การเตรียมความพร้อมพื้นฐาน



1. ติดตั้งสภาพแวดล้อม IDE

ดาวน์โหลดและติดตั้ง Visual Studio Code (รองรับ Ubuntu, Windows, MacOS และ WSL2 Remote)



2. เพิ่มส่วนขยาย Wokwi

ค้นหาและติดตั้ง 'Wokwi Simulator' จาก VS Code Marketplace เพื่อเปิดใช้งานความสามารถแบบออนไลน์



3. ขอรหัสอนุญาต (License Key)

ขอรับ Wokwi License Key ฟรี 1 ปี ผ่าน wokwi.com/license
เปิด Command Palette พิมพ์คำสั่ง:
> Wokwi: Request License

ไฟล์แกนหลักสองส่วนที่ขาดไม่ได้

โทโพโลยีฮาร์ดแวร์ (diagram.json)

```
{  
  "type": "wokwi-arduino-uno",  
  "id": "uno",  
  "attrs": {}  
}
```

ทำหน้าที่เสมือนบอร์ดจริง
กำหนดไมโครคอนโทรลเลอร์ อุปกรณ์ต่อพ่วง
และสายไฟเสมือนที่เชื่อมต่อพินต่างๆ

การจับคู่เฟิร์มแวร์ (wokwi.toml)

```
[wokwi]  
version=1  
firmware='build/arduino-lcd.hex'  
elf='build/arduino-lcd.elf'
```

ทำหน้าที่เสมือนคำสั่งบูตโหลดเดอร์
บอกเอนจิน Wokwi ว่าจะหาไฟล์ไบนารี (.hex/.bin)
และสัญลักษณ์ดีบั๊ก (.elf) ได้ที่ไหน

กระบวนการทำงาน 1: Arduino CLI (เส้นทางมาตรฐาน)

Ubuntu / WSL2 Terminal

การติดตั้ง

```
curl -fsSL https://raw.githubusercontent.com/arduino/arduino-cli/master/install.sh | sh
```

ดาวน์โหลดและติดตั้ง CLI แบบสแตนด์อะโลน โดยไม่ต้องพึ่งพา Arduino IDE แบบเดิม

การกำหนดค่าบอร์ด

```
arduino-cli core update-index  
arduino-cli core install arduino:avr
```

ติดตั้งแพ็คเกจบอร์ดที่จำเป็นสำหรับการคอมไพล์โค้ดให้ Arduino Uno, Nano และ Mega

การจัดการไลบรารี

```
arduino-cli lib install "LiquidCrystal I2C"
```

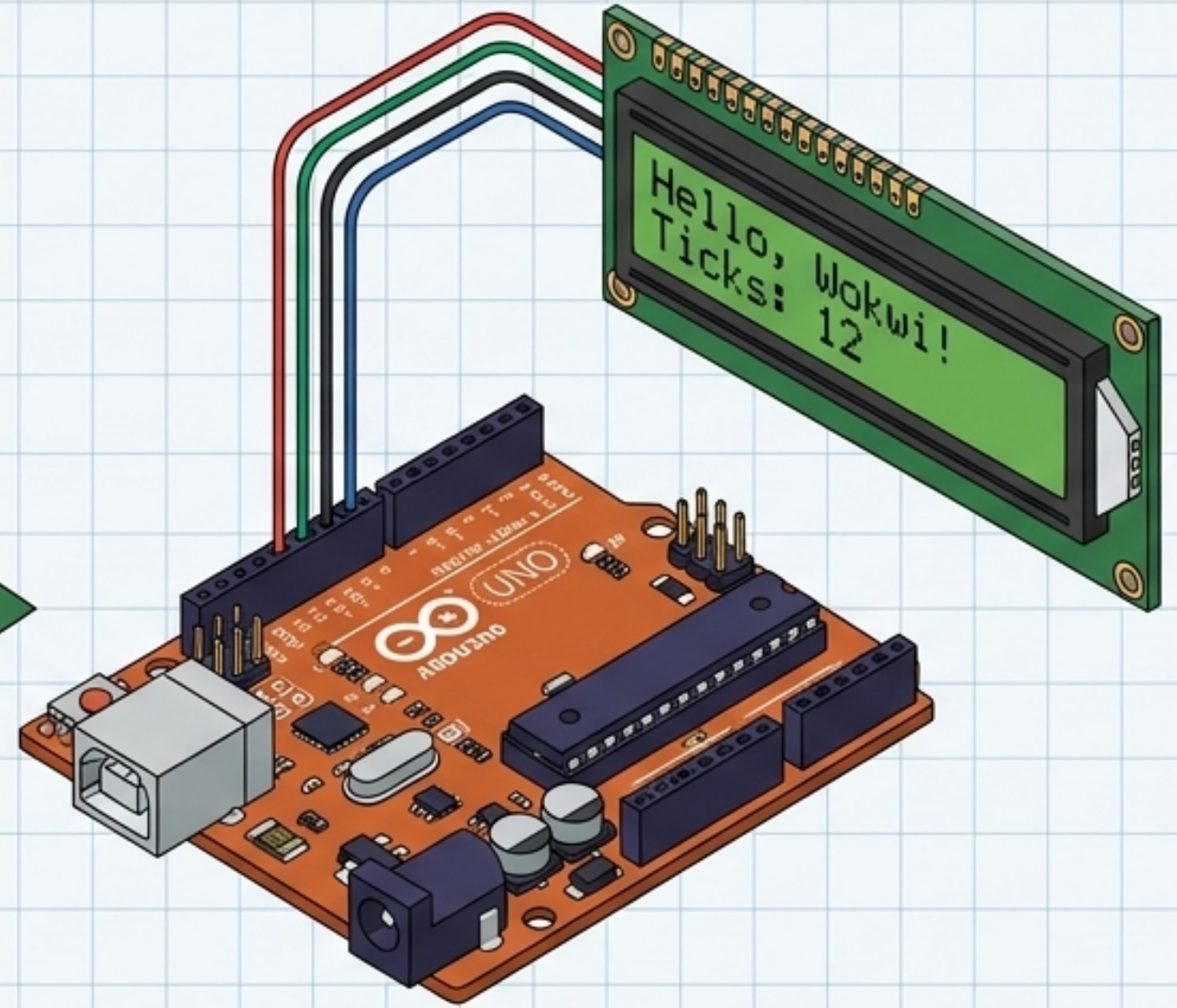
ติดตั้งไลบรารี C++ ภายนอกได้โดยตรงจากเทอร์มินัล

Arduino CLI ภาคปฏิบัติ: LCD Hello World

.vscode/tasks.json

```
arduino-cli compile  
--fqbn arduino:avr:uno  
--output-dir ./build
```

เรียกใช้งานทันทีด้วยคีย์ลัด Ctrl+Shift+B
เพื่อคอมไพล์สเก็ท .ino และนำไฟล์ไบนารี
ไปเก็บไว้ที่โฟลเดอร์ ./build



การเชื่อมต่อ Serial Monitor สมบูรณ์ที่ Baud rate 115200

กระบวนการทำงาน 2: PlatformIO (เส้นทางขั้นสูง)

โครงสร้างไดเรกทอรีของ PIO



ต่างจากสเก็ทซ์ Arduino มาตรฐาน
PIO ยังคงใช้โครงสร้างไดเรกทอรี C++
ที่เป็นโมดูลาร์และเป็นมืออาชีพ

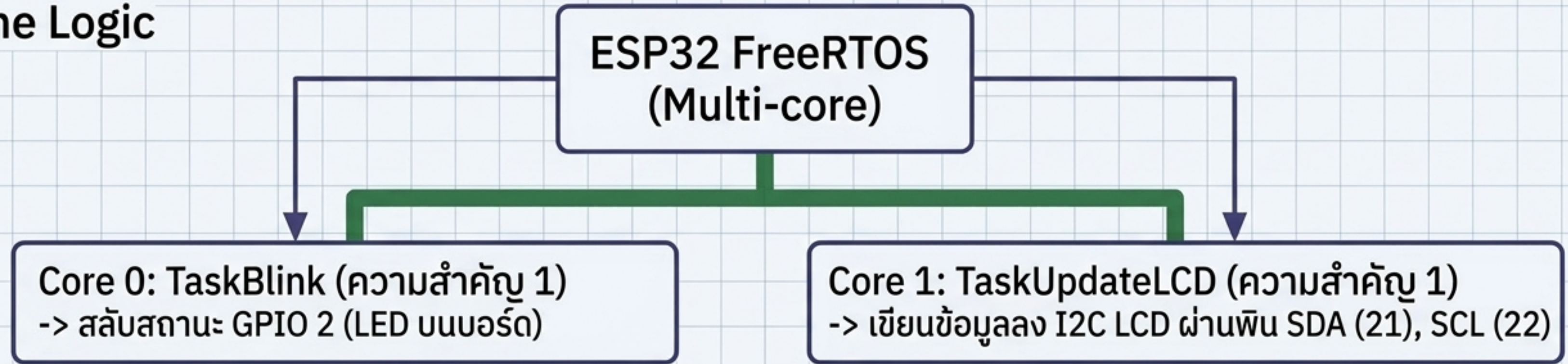
ไฟล์กำหนดค่าศูนย์กลาง

```
[env:uno]
platform = atmelavr
board = uno
framework = arduino
```

ไฟล์เดียวจัดการได้ทั้งหมด
`platformio.ini` ควบคุมบอร์ดเป้าหมาย
เฟรมเวิร์ก และไลบรารีในบล็อกเดียว

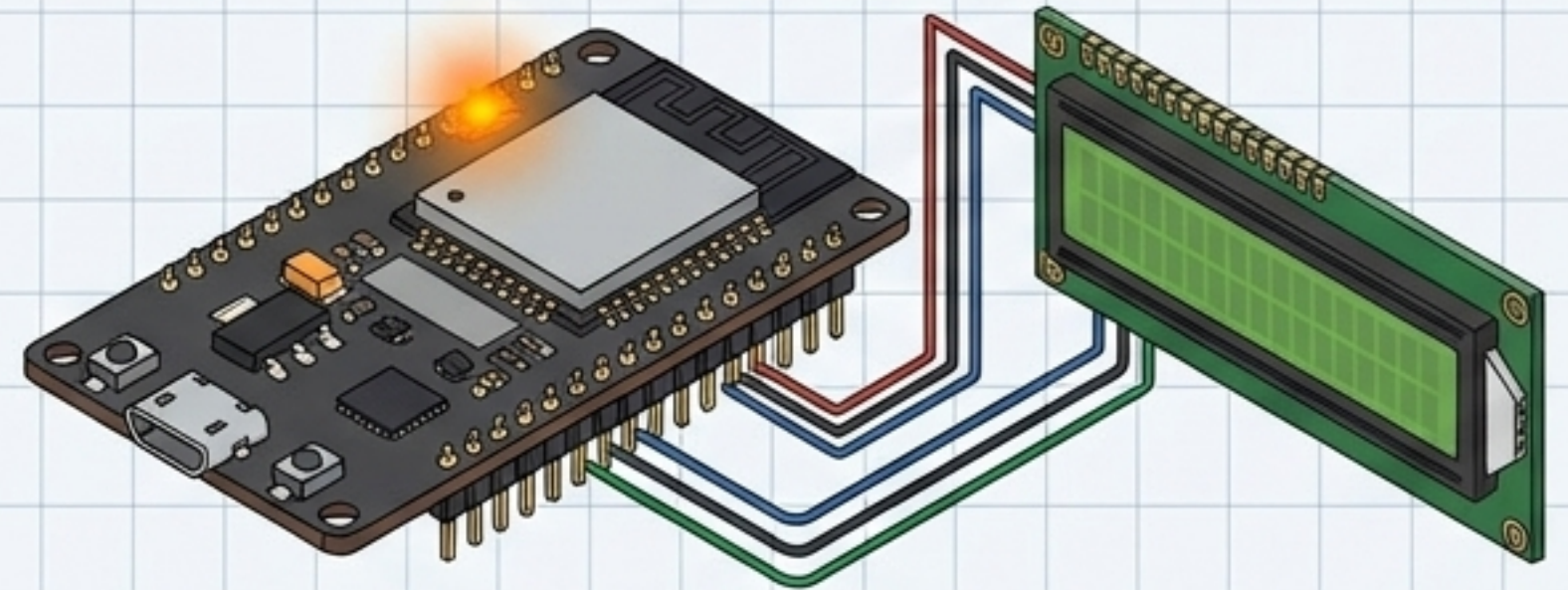
PlatformIO ภาคนิปฏิบัติ: ESP32 + FreeRTOS

The Logic



The Implementation

```
xTaskCreatePinnedToCore(...)
```



คอมไพล์ส่อาดและสมบูรณด้วยคำสั่งเดียว: `pio run`

กระบวนการทำงาน 3: AVR LibC (เส้นทางระดับ Bare-Metal)

1. เครื่องมือคอมไพล์ (The Toolchain)

```
sudo apt install gcc-avr binutils-avr avr-libc avrdude
```

ติดตั้งชุดคอมไพเลอร์ GNU พื้นฐานสำหรับชิปตระกูล Atmel เพื่อควบคุมฮาร์ดแวร์โดยตรง



2. การคอมไพล์ (The Compilation)

```
avr-gcc -mmcu=atmega328p -o main.elf src/main.c
```

แปลงซอร์สโค้ด main.c ดิบๆ ให้เป็นไฟล์รูปแบบ .elf ที่สามารถเรียกทำงานได้



3. การสร้างไฟล์ไบนารี (The Binary Extraction)

```
avr-objcopy -O ihex main.elf main.hex
```

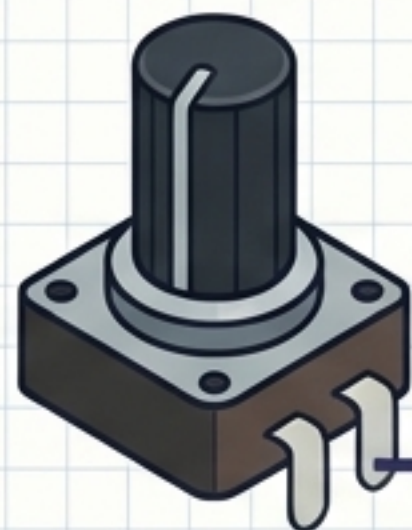
สกัดและลบข้อมูลเมตาดาต้าออก เพื่อสร้างไฟล์ .hex บริสุทธิ์สำหรับให้ wokwi.toml ดึงไปจำลองในชิปเสมือน

AVR ภาคปฏิบัติ: จำลองตรรกะไฟฟ้าด้วย ADC & PWM

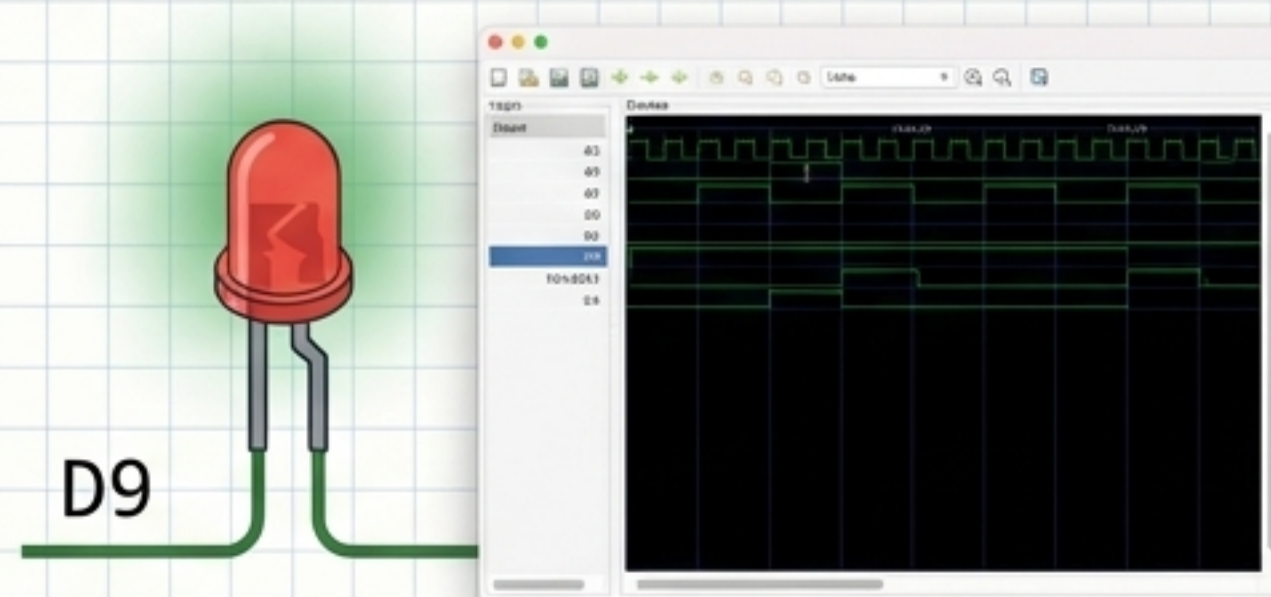
ข้อมูลเข้า:
แอนะล็อก (ADC)

การประมวลผล:
ATmega328P
Timer1

ผลลัพธ์และการวิเคราะห์:
GTKWave



A0



D9

โพเทนชิโอมิเตอร์
ทำหน้าที่แบ่งแรงดัน
0V-5V ส่งเข้าพิน A0

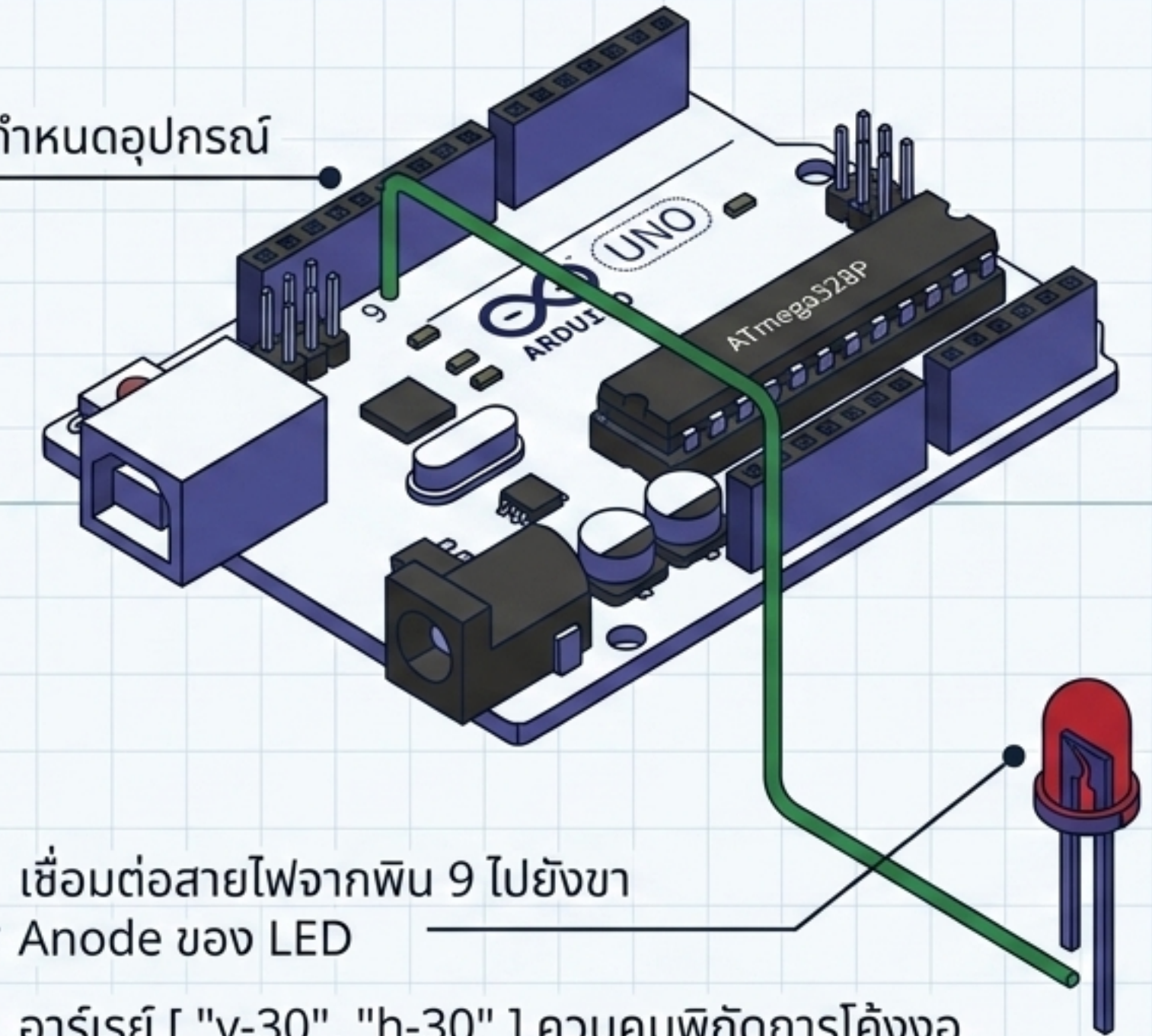
โค้ด Bare-metal
อ่านข้อมูล 10-bit ADC และ
และสั่งงานฮาร์ดแวร์ Timer1
เพื่อสร้างสัญญาณ PWM
ออกทางพิน D9

Wokwi ไม่ได้จำลองแค่พิกเซล
แต่จำลองตรรกะทางไฟฟ้าตาม
เวลาจริงผ่านไฟล์ .vcd
(Value Change Dump)

การแปลความหมายจากโค้ดสู่ฮาร์ดแวร์ (diagram.json)

```
1 {  
  "type": "wokwi-led",  
  "id": "led1",  
  "attrs": {  
    "color": "red"  
  }  
}  
2 [  
  "uno:9",  
  "led1:A",  
  "green",  
  [ "v-30",  
    "h-30"  
  ]  
]
```

กำหนดอุปกรณ์



อาร์เรย์ ["v-30", "h-30"] ควบคุมพิกัดการโค้งงอ
แนวตั้งและแนวนอนของสายไฟสีเขียวบนกริดเสมือน

ตารางเปรียบเทียบเครื่องมือ: เลือกเส้นทางของคุณ

	Arduino CLI	PlatformIO	AVR-GCC
กลุ่มเป้าหมาย	ผู้เริ่มต้น นักการศึกษา และการสร้างต้นแบบอย่าง รวดเร็ว	นักพัฒนาขั้นสูง โปรเจกต์ที่มีหลายไลบรารี และผู้ใช้ RTOS	ผู้เชี่ยวชาญด้านฮาร์ดแวร์ นักศึกษาระบบสมองกลฝังตัว และการปรับแต่งระดับลึก
ความซับซ้อน ในการตั้งค่า	ปานกลาง (ต้องติดตั้งคอร์และ ไลบรารีแบบแมนนวล)	ต่ำ (จัดการอัตโนมัติผ่าน สคริปต์และไฟล์ .ini)	สูง (ต้องจัดการ Makefile และชุดคำสั่งคอมไพล์เอง)
การกำหนดค่าหลัก	<code>tasks.json</code> + <code>arduino-cli.yaml</code>	<code>platformio.ini</code>	พิมพ์คำสั่ง Terminal โดย ตรง หรือเขียน Makefile

ทั้งสามเส้นทางนำไปสู่เอนจินจำลอง Wokwi ตัวเดียวกัน
เลือกเครื่องมือที่เหมาะสมกับระดับความเป็นนามธรรมที่คุณต้องการ

บทสรุปและแหล่งเรียนรู้เพิ่มเติม



เจาะลึกการใช้ ESP32 + Wokwi

อ่านบทความตอนที่ 1 & 2 ของซีรีส์ ESP32 ขั้นสูง

iot-kmutnb.github.io/blogs/esp32/...



การใช้ PlatformIO Extension อย่างเชี่ยวชาญ

คู่มือฉบับเต็มเกี่ยวกับการจัดการ PIO ใน VS Code

iot-kmutnb.github.io/blogs/esp32/esp32_vscode_pio/



เอกสารประกอบของ Wokwi อย่างเป็นทางการ

สำรวจฮาร์ดแวร์และส่วนประกอบอิเล็กทรอนิกส์ทั้งหมดที่รองรับ

docs.wokwi.com